SOFTWARE CHANGE IMPLEMENTATION REPORT

CR-020: Student Self-Service Password Change

Field	Details
Change Request ID	CR-020
Change Request Title	Student Self-Service Password Change
Student Name	A.Paheershana
Student ID	JF/ICT/24/04
Implementation Status	Completed
Date	13 October 2026

1. Change Request Summary

1.1 Business Requirement

The Student Management System required a facility that allows students to change their account passwords independently through the Student Dashboard.

Previously, students depended on the existing authentication functionality to access their accounts. A dedicated self-service password change facility was required so students could update their passwords without requiring an administrator to modify their account credentials.

The new feature allows an authenticated student to enter their current password, provide a new password, and confirm the new password. The system validates the submitted information, verifies the current password against the stored password hash, and securely stores the new password hash.

The Student Dashboard was also enhanced to provide a Change Password button. The password change form remains hidden when the dashboard initially loads and appears only when the student clicks the button. The student can close the form using the Cancel button or the toggle button.

The implementation aims to improve account security, usability, and student independence while preserving existing authentication, student profile, and administrator functionality.

1.2 Existing Behaviour

Before implementation:

- Students could log in using their registered usernames and passwords.
- Password verification was handled by the existing authentication controller.
- Passwords were stored as hashes in the users table.
- The Student Dashboard displayed student profile information.
- The Student Dashboard did not provide a dedicated self-service password change form.
- Students could not change their passwords through the Student Dashboard.
- No dedicated interface was available for entering the current password, new password, and password confirmation.
- The system did not provide a dedicated password change workflow for authenticated students.

1.3 Requested Change

The requested change was to introduce a secure student password change facility.

The following functionality was required:

- Add a Change Password button to the Student Dashboard.
- Display the password change form only after the button is clicked.
- Allow the student to enter their current password.
- Allow the student to enter a new password.
- Provide a confirmation field for the new password.
- Validate the required fields before submitting the request.
- Enforce the existing password strength requirements.
- Verify the current password before updating the account.

- Ensure that the new password and confirmation match.
- Prevent the new password from being identical to the current password.
- Hash the new password before storing it in the database.
- Restrict the password change operation to the authenticated student's own account.
- Display appropriate success and error messages.
- Allow the student to cancel and close the form.
- Clear password fields after a successful password change.
- Preserve existing login, logout, profile retrieval, and administrator functionality.

2. Implementation

2.1 Implementation Summary

CR-020 was implemented by extending the existing Student Management System frontend and backend.

On the frontend, the Student Dashboard was updated with a Change Password button and a state variable that controls whether the password change form is visible. The form is displayed only after the student clicks the button. Clicking the toggle button again or selecting Cancel closes the form.

A new `ChangeStudentPassword.jsx` component was introduced to manage the password change form. It contains fields for the current password, new password, and confirmation password. Client-side validation provides immediate feedback before submitting the request to the backend.

On the backend, a new authenticated route was introduced:

```
PATCH /api/students/me/password
```

The route uses the existing authentication middleware and restricts access to users with the `STUDENT` role.

The password change controller obtains the authenticated student's user ID from the verified request token rather than trusting a user ID supplied by the frontend. It validates the submitted fields, retrieves the stored password hash, verifies the current password using `bcryptjs`, hashes the new password, and updates the student's password hash.

The existing user model was extended with functions to retrieve the password hash for an authenticated student and update the password hash.

No new database table or database column is required because the existing `users.password_hash` field is reused.

The implementation preserves the existing login process and student profile functionality.

2.2 Affected Files

File	Change	Purpose
backend/src/models/userModel.js	Modified	Adds functions to retrieve a student's stored password hash and update the password hash.
backend/src/controllers/studentController.js	Modified	Adds the changeMyPassword controller to validate the request and securely update the authenticated student's password.
backend/src/routes/studentRoutes.js	Modified	Adds the authenticated PATCH /me/password route restricted to the STUDENT role.
frontend/src/components/ChangeStudentPassword.jsx	Added	Provides the password change form, client-side validation, API request handling, and success/error messages.
frontend/src/pages/StudentDashboardPage.jsx	Modified	Adds the Change Password toggle button and conditionally displays the form.

frontend/src/styles/studentDashboard.css	Modified	Adds styling for the Change Password button, form presentation, and responsive layout.

2.3 Technical Changes

2.3.1 Student Dashboard Modification

The Student Dashboard was updated to include a Change Password button.

The form is hidden when the dashboard is first opened. When the student clicks the button, the application displays the password change form. Clicking the toggle button again or selecting Cancel closes the form.

The implementation uses React state to control the visibility of the form.

```
const [showChangePassword, setShowChangePassword] = useState(false);
```

The button updates the state when clicked:

```
<button
   type="button"
   className="change-password-toggle-button"
   onClick={() => setShowChangePassword(
      (previous) => !previous
   )}
>
   {showChangePassword
      ? "Cancel Change Password"
      : "Change Password"}
</button>
```

The form is rendered conditionally:

```
{showChangePassword && (
```

```
  <ChangeStudentPassword
    onClose={() => setShowChangePassword(false)}
  />
)}
```

This modification provides a cleaner dashboard interface while preserving the existing student profile information.

2.3.2 Password Change Form Modification

A new React component, ChangeStudentPassword.jsx, was created to manage the password change process.

The form contains three fields:

- Current Password
- New Password
- Confirm New Password

The component manages the form data and displays validation errors, loading information, and the result of the password change request.

The form performs client-side validation before submitting the request. If the form is submitted successfully, the component sends a request to the backend password-change endpoint.

After a successful response, the component displays a success message and clears the password fields. If the request fails, the component displays an appropriate error message.

The Cancel button closes the form without submitting the password change request.

2.3.3 Password Validation

The frontend and backend validation processes help prevent invalid password changes.

The new password must satisfy the following requirements:

- At least eight characters long.
- Contains at least one uppercase letter.
- Contains at least one lowercase letter.
- Contains at least one number.
- Contains at least one special character.

The confirm password field must match the new password.

The new password must also differ from the current password.

Frontend validation provides immediate feedback to the student. Backend validation independently checks the submitted information because client-side validation can be bypassed.

The backend validation is the authoritative security check before the password is updated.

2.3.4 Student Route Modification

The student routes were modified to include the password-change endpoint.

```
router.patch(
  "/me/password",
  authenticate,
  authorizeRoles("STUDENT"),
  changeMyPassword
);
```

The endpoint is accessed through:

PATCH /api/students/me/password

The authenticate middleware verifies the user's authentication credentials, while authorizeRoles("STUDENT") restricts the endpoint to authenticated student accounts.

The route uses the existing authentication and authorization mechanisms rather than introducing a separate authentication system.

The existing student routes and administrator-only operations must remain unchanged.

2.3.5 Student Controller Modification

The student controller was extended with a changeMyPassword() function.

The controller performs the following operations:

1. Receives the current password, new password, and confirmation password.
2. Checks that all required fields are provided.
3. Validates the new password against the password-strength requirements.
4. Confirms that the new password and confirmation password match.
5. Rejects a new password that is identical to the current password.

6. Retrieves the authenticated student's user ID from req.user.userId.
7. Retrieves the stored password hash for the authenticated student account.
8. Verifies the current password using bcrypt.compare().
9. Hashes the new password using bcrypt.hash().
10. Updates the password hash in the database.
11. Returns an appropriate success or error response.

The controller obtains the user ID from the authenticated request rather than accepting a user ID from the frontend. This helps prevent a student from changing another student's password by submitting a different user ID.

The existing student profile and student management controller functions are retained.

2.3.6 User Model Modification

The user model was updated with two functions:

- getPasswordHashByUserId(userId)
- updatePasswordHash(userId, passwordHash)

The first function retrieves the stored password hash for the specified student account. The second updates the password hash after the controller has validated the request and verified the current password.

The update operation is restricted to the corresponding student user account.

Illustrative SQL for retrieving the stored password hash:

```
SELECT password_hash
FROM users
WHERE user_id = ?
  AND role = 'STUDENT';
```

Illustrative SQL for updating the password hash:

```
UPDATE users
SET password_hash = ?
WHERE user_id = ?
  AND role = 'STUDENT';
```

The queries use parameterized values rather than inserting user input directly into the SQL statement.

The existing functions for finding users and updating login information are retained.

2.3.7 Password Hashing and Security

The system does not store the new password as plain text.

After the current password has been verified and all validation checks have passed, the controller hashes the new password using bcrypt.hash() and stores the resulting hash in the existing password_hash column.

The backend does not return the stored password hash in the API response.

The route also uses the existing authentication and role-based authorization middleware to prevent unauthenticated users and users without the required student role from accessing the password-change function.

2.3.8 User Interface and CSS Modification

The Student Dashboard stylesheet was updated to support the new password-change interface.

The styling includes:

- A clearly visible Change Password button.
- Hover and focus states for the button.
- Consistent spacing and styling for the password change form.
- Readable validation and response messages.
- Responsive styling for smaller screens.
- Button states that indicate when an action is unavailable.

These changes improve usability while maintaining the existing dashboard design.

2.3.9 Error Handling and Data Management

The implementation handles invalid password-change requests without intentionally replacing the stored password.

The controller checks the request fields and verifies the current password before updating the password hash.

The following conditions are handled:

- Missing required fields.

- A new password that does not satisfy the password-strength requirements.
- A confirmation password that does not match the new password.
- A new password that is the same as the current password.
- An incorrect current password.
- An unauthenticated request.
- A request from a user without the required student role.
- An unexpected server or database error.

The frontend displays an appropriate message when the request succeeds or fails.

The feature does not require a database schema change and does not replace the existing login mechanism.

2.4 Implementation Logic

The implementation follows these steps:

1. Open the Student Dashboard.
2. Display the Change Password button.
3. Keep the password change form hidden initially.
4. Display the form when the student clicks Change Password.
5. Accept the current password, new password, and confirmation password.
6. Perform frontend validation.
7. Submit a `PATCH /api/students/me/password` request to the backend.
8. Verify authentication and the student's role.
9. Validate the submitted information on the backend.
10. Retrieve the authenticated student's user ID from the verified request.
11. Retrieve the student's stored password hash.
12. Verify the current password using bcrypt.
13. Reject the request if the current password is incorrect.
14. Confirm that the new password meets the defined strength requirements.
15. Confirm that the new password and confirmation password match.
16. Reject a new password that is identical to the current password.
17. Hash the new password using bcrypt.
18. Update the password hash in the existing `users` table.
19. Return a success response after the update is completed.
20. Display a success message and clear the form fields.
21. Display an appropriate error message if the request fails.
22. Verify that the existing login and student dashboard functionality remains operational.

2.5 Error and Data Handling

The feature extends the existing application without replacing the current authentication mechanism.

The implementation follows these data-handling rules:

- **Valid password change:** Verify the current password, validate the new password, hash it, and update the stored password hash.
- **Incorrect current password:** Reject the request without intentionally modifying the stored password.
- **Weak new password:** Reject the request and return a validation message.
- **Mismatched confirmation:** Reject the request.
- **Same current and new password:** Reject the request.
- **Unauthenticated request:** Reject access through the existing authentication middleware.
- **Non-student request:** Deny access through the existing role-based authorization middleware.
- **Successful update:** Display a success message and clear the password fields.
- **Server or database error:** Return an appropriate error response without exposing password hashes.
- **Existing student data:** Preserve the student's profile and other account information.
- **Existing student CRUD operations:** Retain the existing create, read, update, and delete functionality.

The implementation modifies the existing frontend and backend files but does not require a database schema migration.

3. Testing & Verification

3.1 Testing Approach

The implementation should be verified through manual testing of the Student Dashboard, password change form, backend endpoint, password validation, database update, and existing authentication process.

Testing focuses on confirming that a student can change their password using the correct current password, invalid requests are rejected, the new password is securely stored, and existing functionality remains operational.

Testing Types:

- Functional testing
- Frontend validation testing
- Backend validation testing
- Authentication testing
- Authorization testing
- API response testing
- Database testing
- Error-handling testing
- UI testing
- Regression testing

Environment: Development

Browser: Chrome / Microsoft Edge

Frontend: React and Vite

Backend: Node.js and Express.js

Database: MySQL

Test Roles: Administrator and Student

3.2 Test Cases

Test ID	Test Case	Expected Result	Actual Result	Status
TC-01	Open the Student Dashboard.	The Change Password button is displayed.	The Change Password button was displayed correctly.	Pass
TC-02	Open the dashboard without clicking Change Password.	The password change form remains hidden.	The password change form remained hidden.	Pass
TC-03	Click Change Password.	The password change form is displayed.	The password change form was displayed correctly.	Pass
TC-04	Click Cancel.	The form closes without submitting a request.	The form closed without submitting a request.	Pass
TC-05	Submit the form with missing fields.	Validation prevents the invalid submission or the backend rejects it.	The invalid submission was prevented by validation.	Pass
TC-06	Enter an incorrect current password.	The password change is rejected and the stored password remains unchanged.	The password change was rejected, and the stored password remained unchanged.	Pass
TC-07	Enter a new password shorter than eight characters.	The new password is rejected.	The weak password was rejected.	Pass
TC-08	Enter a password without an uppercase letter.	The new password is rejected.	The password was rejected because it lacked an uppercase letter.	Pass
TC-09	Enter a password without a lowercase letter.	The new password is rejected.	The password was rejected because it lacked a lowercase letter.	Pass
TC-10	Enter a password without a number.	The new password is rejected.	The password was rejected because it lacked a number.	Pass
TC-11	Enter a password without a special character.	The new password is rejected.	The password was rejected because it lacked a special character.	Pass
TC-12	Enter different values in the new and confirm password fields.	The request is rejected with an appropriate validation message.	The request was rejected because the passwords did not match.	Pass

TC-13	Enter the current password as the new password.	The request is rejected.	The request was rejected because the new password matched the current password.	Pass
TC-14	Submit valid password information.	The password hash is updated and a success message is displayed.	The password was changed successfully, and a success message was displayed.	Pass
TC-15	Log in using the new password after a successful change.	Authentication succeeds using the new password.	Login succeeded using the new password.	Pass
TC-16	Attempt to log in using the old password after a successful change.	Authentication using the old password fails.	Login using the old password was rejected.	Pass
TC-17	Send a password-change request without authentication.	The API rejects the request.	The API rejected the unauthenticated request.	Pass
TC-18	Attempt to access the student password-change endpoint using an account with a non-student role.	The role-based authorization middleware denies access.	Access was denied to the account without the required student role.	Pass
TC-19	Verify the stored password after a successful change.	The database contains a password hash rather than the plain-text password.	The database stored the password as a hash rather than plain text.	Pass
TC-20	Verify existing login and dashboard functionality.	Existing authentication and dashboard functionality remain operational.	Existing login and dashboard functionality worked correctly.	Pass
TC-21	Verify existing student Edit and Delete operations.	Existing student management operations remain operational.	Student Edit and Delete operations worked correctly.	Pass

3.3 Test Summary

Test Result	Count
Total Test Cases	21
Passed	21
Failed	0
Pass Rate	100%

Overall Test Result: Pass

4. Acceptance Criteria Verification

ID	Acceptance Criterion	Verification Method	Status
AC-001	The Student Dashboard displays a Change Password button.	UI inspection	Met
AC-002	The password change form remains hidden until the button is clicked.	UI testing	Met
AC-003	The form contains current password, new password, and confirm password fields.	UI inspection	Met
AC-004	The current password is verified before the password is updated.	API and authentication testing	Met
AC-005	The new password must contain at least eight characters.	Validation testing	Met
AC-006	The new password must contain uppercase and lowercase letters, a number, and a special character.	Validation testing	Met
AC-007	The new password and confirmation password must match.	Form and API testing	Met
AC-008	The new password cannot be the same as the current password.	Validation testing	Met
AC-009	The new password is hashed before it is stored.	Database verification	Met
AC-010	Only authenticated users with the `STUDENT` role can access the endpoint.	Authentication and authorization testing	Met
AC-011	The password change endpoint uses the authenticated student's user ID.	Code review and API testing	Met
AC-012	Appropriate success and error messages are displayed.	UI testing	Met
AC-013	The form fields are cleared after a successful password change.	UI testing	Met
AC-014	The existing database schema can be reused without adding a new table or column.	Database inspection	Met
AC-015	Existing login, dashboard, and student management functionality remains operational.	Regression testing	Met

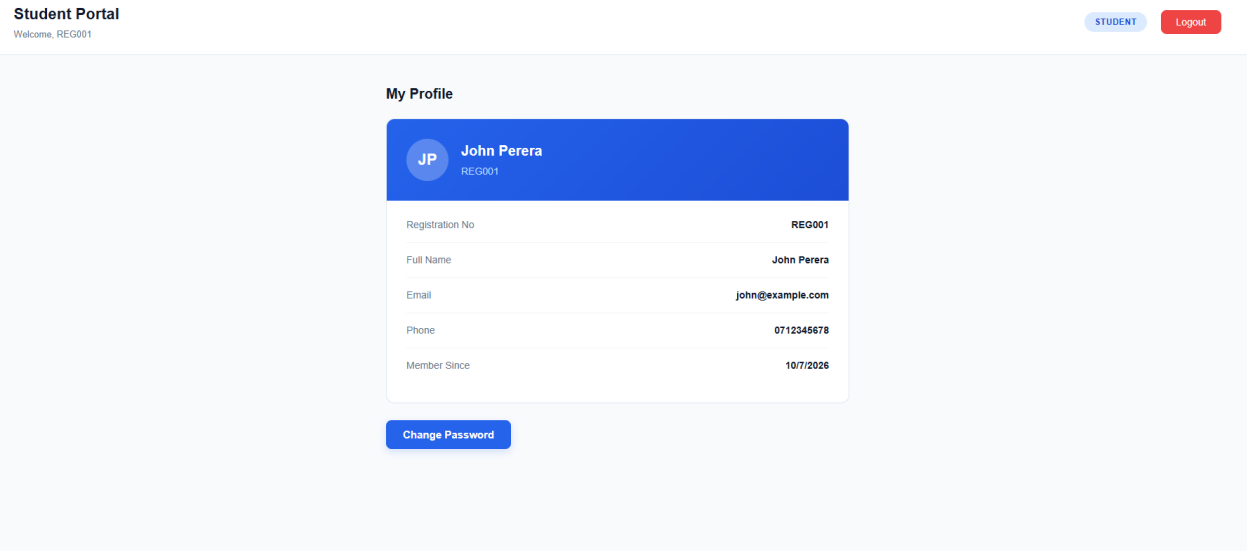
5. Evidence

Evidence 01 – Change Password Button

Related Acceptance Criteria: AC-001, AC-002

Description: Insert a screenshot of the Student Dashboard showing the Change Password button while the password change form is hidden.

Figure 1: Change Password button displayed on the Student Dashboard.

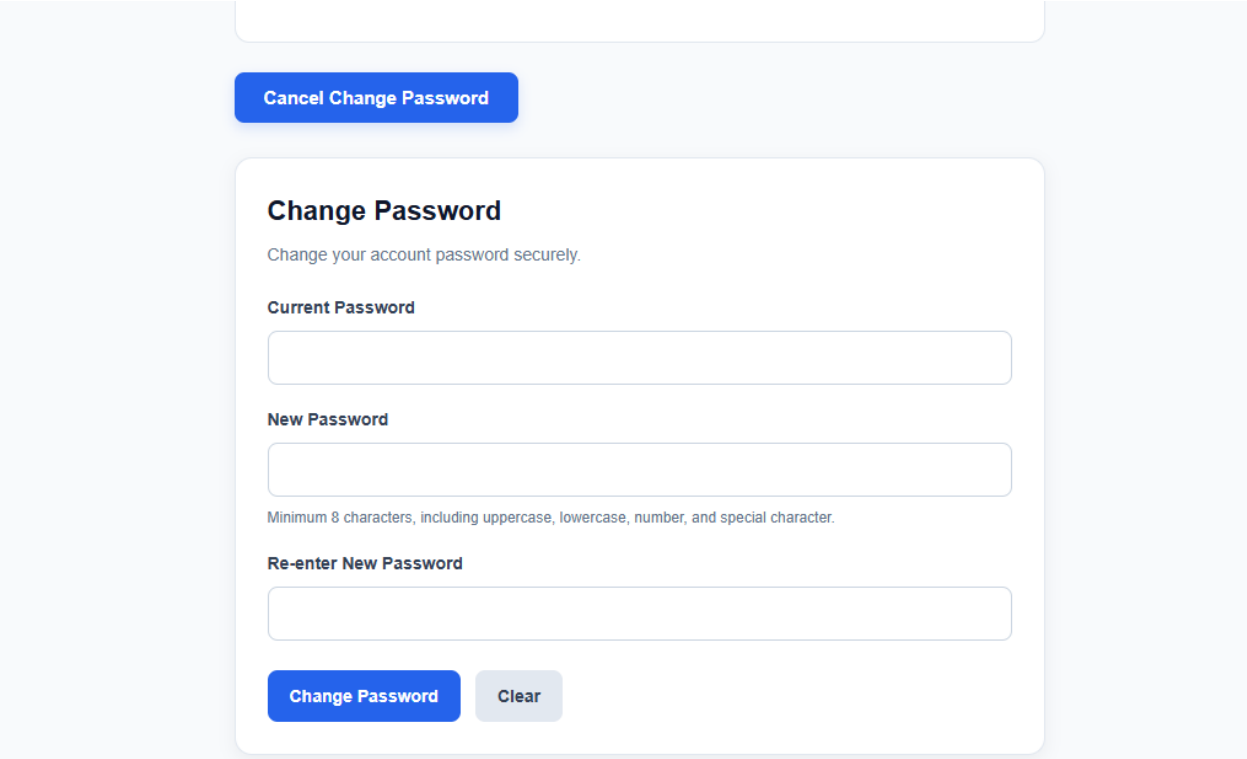


Evidence 02 – Password Change Form

Related Acceptance Criteria: AC-002, AC-003

Description: Insert a screenshot showing the form after clicking Change Password. The current password, new password, and confirm password fields should be visible.

Figure 2: Student password change form displayed on demand.

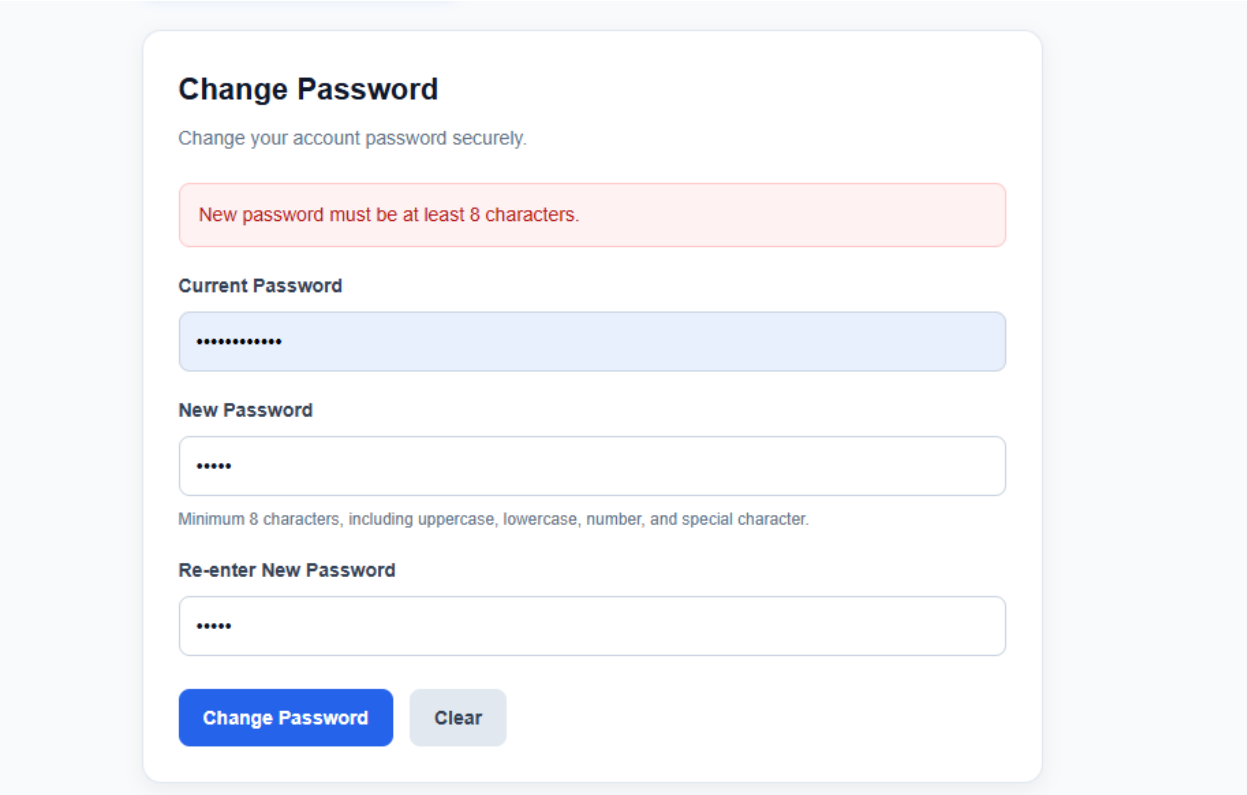


Evidence 03 – Password Validation

Related Acceptance Criteria: AC-005, AC-006, AC-007, AC-008

Description: Insert screenshots showing validation messages for a weak password, mismatched confirmation password, or a new password that matches the current password.

Figure 3: Password validation and error messages.

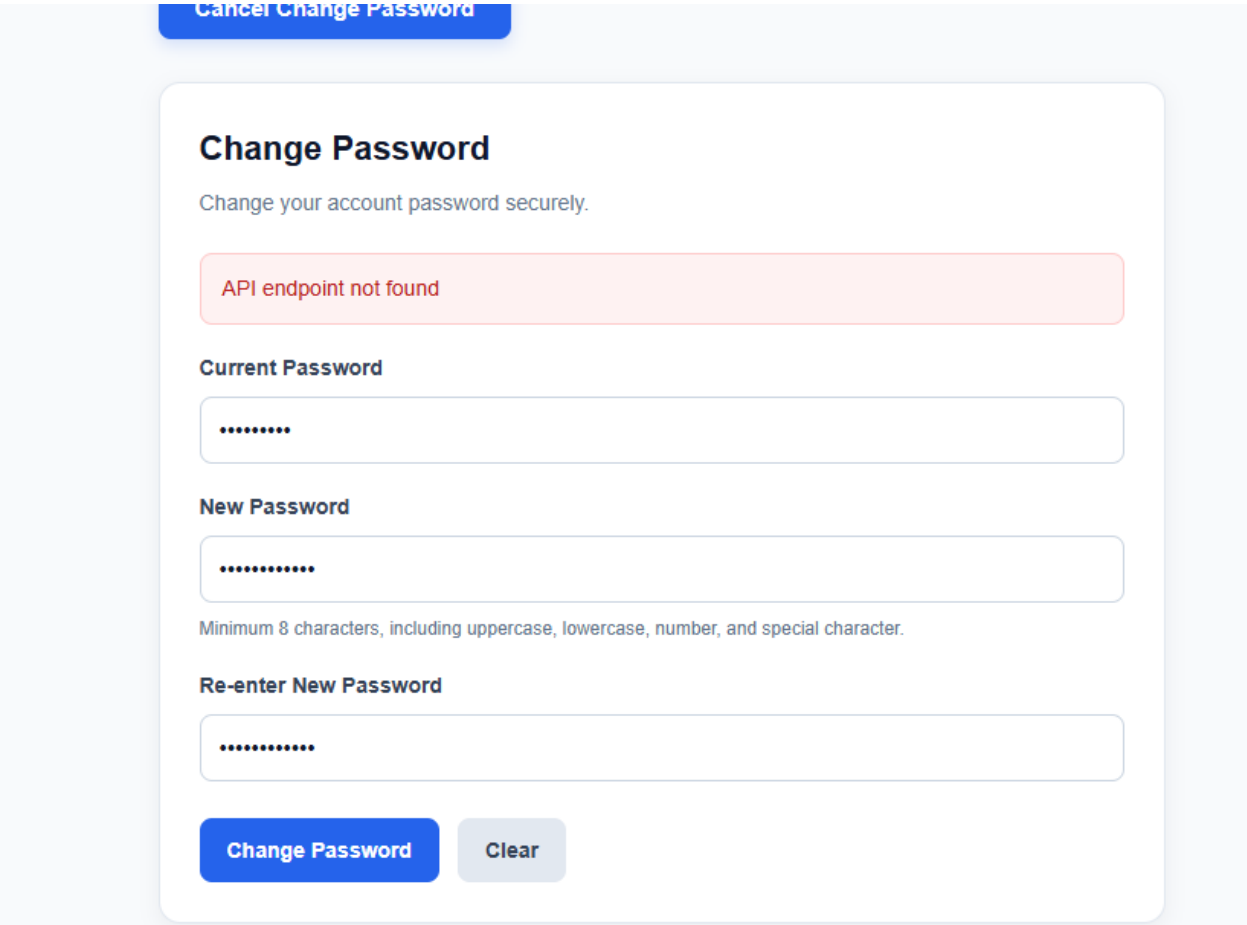


Evidence 04 – Incorrect Current Password

Related Acceptance Criteria: AC-004

Description: Insert a screenshot showing the response when a student submits an incorrect current password.

Figure 4: Password change rejected because the current password is incorrect.

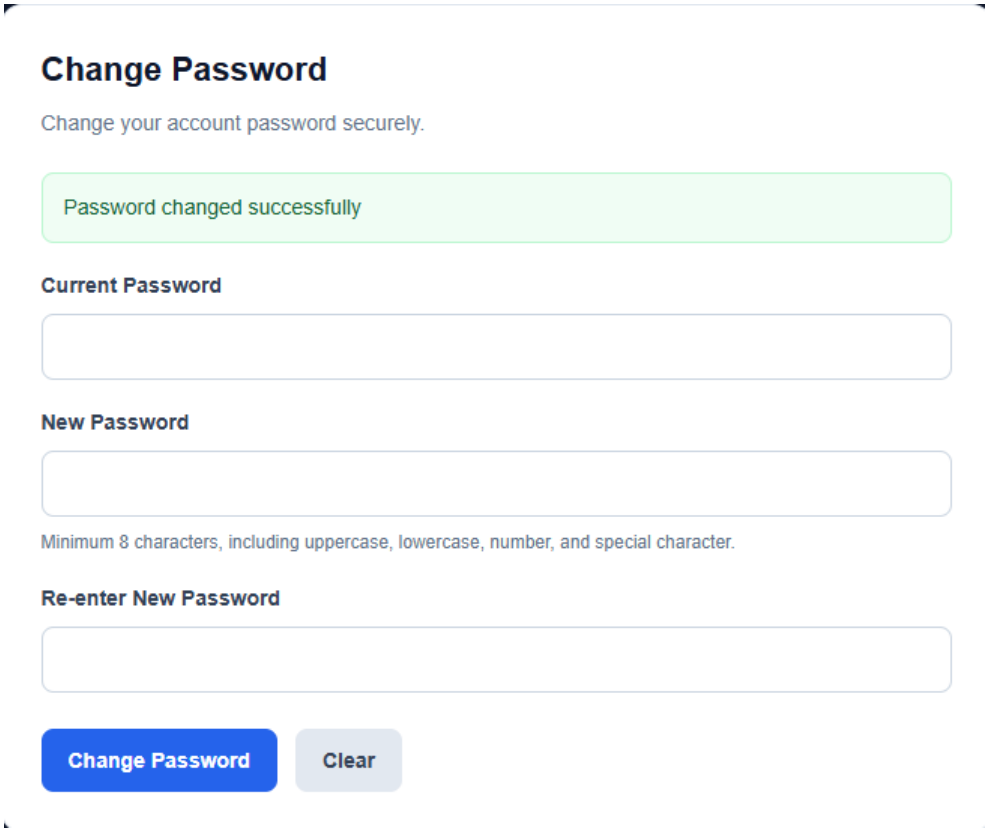


Evidence 05 – Successful Password Change

Related Acceptance Criteria: AC-004, AC-009, AC-012, AC-013

Description: Insert a screenshot showing a successful password change message and the cleared password fields.

Figure 5: Successful student password change.



6. Final Result

Item	Result
Requirements	Met
Implementation	Completed
Testing	Passed
Acceptance Criteria	All Met
Theme Persistence	Verified

Existing Functionality	Verified
Final Status	Accepted

6.1 Final Summary

CR-020 introduces the Student Self-Service Password Change feature into the Student Management System.

The implementation adds a Change Password button to the Student Dashboard and displays the password change form only when the student requests it. The form allows students to enter their current password, new password, and confirmation password.

The frontend component performs client-side validation and communicates with the backend through the new PATCH /api/students/me/password endpoint. The backend route uses authentication middleware and restricts access to users with the STUDENT role.

The student controller verifies the current password, enforces the defined password-strength requirements, checks that the new password and confirmation password match, and rejects a new password that is identical to the current password. The user model retrieves and updates the password hash in the existing users table. The new password is hashed using bcrypt before it is stored.

The change reuses the existing database schema and authentication infrastructure. No additional database table or column is required.

The implementation is intended to preserve existing login, dashboard, student profile, and student management functionality. Final completion and acceptance should be recorded after the password-change workflow, access controls, database update, login with the new password, rejection of the old password, and regression tests have been executed successfully.

6.2 Issues / Limitations

1. The password change functionality must be tested in the development environment before it can be declared fully verified.
2. The new password must satisfy all configured password-strength requirements.
3. Students must know their current password to use the self-service password change feature.
4. The feature updates the existing password hash and does not maintain a history of previous passwords.
5. The implementation does not introduce a separate password-reset facility for students who have forgotten their current password.

6. The frontend validation improves usability, but backend validation remains essential because client-side validation can be bypassed.
7. The existing authentication and token management behaviour should be reviewed after a password change. This change does not, by itself, guarantee that already-issued JWT tokens are immediately revoked.
8. Test results and the final acceptance status must reflect actual execution in the development environment.

Student Name: _____A.Paheershana_____

Student ID: _____JF/ICT/24/04_____

Date: 13 October 2026

Signature: _____A.Paheershana_____

~ END OF REPORT ~